API de préstamos — referencia para frontend

Rama `prueba_db_middle`

Proyecto `prestamos_sistema`

1. Cambios que rompen compatibilidad

Usuarios y préstamos (último cambio)

1. **Los mensajes de error de Django y DRF ahora vienen en español** ("Este campo es requerido." en vez de "This field is required."). Si el front compara textos de error en inglés, hay que cambiarlo.
2. **Dar de baja a un usuario con** `PATCH {"status": "INACTIVO"}` ahora sigue la misma regla que el `DELETE`: `400` si tiene recursos sin devolver. Sus reservas sin recoger ya no lo impiden: se cancelan solas.

Módulo préstamos

1. **Todas las listas vienen paginadas** (usuarios, catálogo y préstamos). Antes llegaba un arreglo; ahora llega un objeto y el arreglo está en `results`:

```
{"count": 57, "next": "http://.../?page=2", "previous": null, "results": [ ... ]}
```

   20 por página. `?page=2` pide la siguiente y `?page_size=50` cambia el tamaño (máximo 100).

2. **Ya no se pueden eliminar préstamos**: `DELETE /prestamos/admin/reservas/{id}/` responde `405`. El historial se conserva completo; para cerrar un préstamo sin devolución están `perdido/` (admin) y `cancelar/` (usuario).
3. `POST /prestamos/reservar/` **responde el préstamo completo** (antes solo el `id`).
4. **Reservar tiene reglas nuevas** (ver sección Reservas): con un préstamo vencido no se puede pedir otro, solo un ejemplar por recurso y un tope de préstamos simultáneos.
5. **Reservar ya no entrega el recurso.** `POST /prestamos/reservar/` deja el préstamo en `RESERVADO` (sin `fecha_limite`) y el usuario tiene **24 horas** para recogerlo en el mostrador. Cuando lo recoge, el admin usa `POST .../entregar/` y recién ahí pasa a `ACTIVO` y corren sus días. Si no lo recoge, se cancela sola.
6. **Cancelar solo funciona antes de recoger** (estado `RESERVADO`). Ya no existe el plazo de 30 minutos.
7. **Los préstamos ahora sí pasan a** `VENCIDO` cuando se supera la fecha límite, y hay estados nuevos: `RESERVADO`, `PERDIDO` y `CANCELADO`. El ejemplar tiene uno nuevo: `RESERVADO` (apartado, esperando que lo recojan).

8. **Fechas en hora de Perú**: `fecha_limite` y `fecha_devolucion` se calculan con el día de Lima. `fecha_prestamo` sigue llegando en UTC (con `Z` o con `-05:00`).

Módulo catálogo

Todo lo que es gestión de recursos se movió de `/prestamos/` a un módulo propio, `/catalogo/`. Las rutas viejas ya no existen.

Antes	Ahora
`/prestamos/recursos/`	`/catalogo/recursos/`
`/prestamos/admin/recursos/...`	`/catalogo/admin/recursos/...`
`/prestamos/admin/ejemplares/{id}/`	`/catalogo/admin/ejemplares/{id}/`
`/prestamos/admin/categorias/...`	`/catalogo/admin/categorias/...`

Además:

1. **Dar de baja una ficha ya no es un `PATCH`.** `estado_ficha` es de solo lectura; se usa `POST .../baja/` y `POST .../reactivar/`.
2. `condicion` **del ejemplar ya no es texto libre**: es `BUENO`, `REGULAR` o `DANADO`. El texto libre va en el campo nuevo `observaciones`.
3. **Al crear un ejemplar ya no se manda** `ficha_recurso` en el cuerpo: sale de la URL.
4. `categoria` **es obligatoria** al crear una ficha.
5. **Solo** `PATCH`: los endpoints de detalle del catálogo ya no aceptan `PUT`.

Separación en fichas y ejemplares

1. **Reservar ya no manda** `recurso`**, manda** `ficha_recurso`**.** El usuario elige una ficha del catálogo ("Laptop Dell XPS"), no una unidad física puntual — el backend elige automáticamente cuál ejemplar disponible asignarle.
2. **El catálogo ya no trae** `disponible` **(booleano).** Ahora trae `disponibles` (número), porque una ficha puede tener varias unidades físicas con estados independientes.

2. Autenticación

Todos los endpoints salvo registro y login requieren el header:

```
Authorization: Bearer <access_token>
```

Obtenido con `POST /login/`. El token de acceso dura 15 minutos; usar `POST /login/refresh/` con el `refresh` token para renovarlo.

Los endpoints bajo `/admin/` exigen además que `tipo_usuario` del usuario logueado sea `ADMINISTRATOR`; si no, devuelven `403`.

CORS: la API solo acepta llamadas del navegador desde las direcciones configuradas en el backend (por defecto `http://localhost:5173`, el puerto de Vite). Si el front corre en otro puerto o dominio y la consola del navegador muestra un error de CORS, hay que agregar esa dirección a `CORS_ALLOWED_ORIGINS` en el `.env` del backend.

Olvidé mi contraseña (público)

`POST /users/password/olvide/`

```
{"email": "daniel@unsa.edu.pe"}
```

Siempre responde `200` con el mismo mensaje, exista o no el correo (así nadie puede averiguar qué correos están registrados). Si existe y el usuario está activo, le llega un correo con un enlace al **frontend**:

```
<FRONTEND_URL>/restablecer-password?uid=MTI&token=cd1x2-...
```

El front necesita una pantalla en esa ruta que lea `uid` y `token` de la URL, pida la contraseña nueva y llame a:

`POST /users/password/restablecer/`

```
{"uid": "MTI", "token": "cd1x2-...", "password_nueva": "OtraClave456!"}
```

Responde `200` y cierra todas las sesiones abiertas de ese usuario; luego debe iniciar sesión normalmente. El enlace vence en **1 hora** y sirve una sola vez. Un enlace inválido o vencido responde `400` en `token`; una contraseña débil, `400` en `password`.

En desarrollo los correos no se envían: se imprimen en la terminal donde corre el servidor.

3. Usuarios (admin)

`GET, POST /users/admin/usuarios/`

`GET, PATCH, DELETE /users/admin/usuarios/{id}/`

`GET` trae un usuario por su `id` (por ejemplo, el `"usuario": 5` de un préstamo, o para llenar el formulario de edición).

Filtros del listado, combinables:

- `?q=` — DNI o CUI exacto, o parte del nombre, apellido o email (`?q=daniel perez`: cada palabra debe aparecer en alguno de ellos).
- `?tipo_usuario=` `TEACHER`, `STUDENT`, `ADMINISTRATIVE_STAFF` o `ADMINISTRATOR`.
- `?status=` `ACTIVO`, `INACTIVO` o `SUSPENDIDO`.

Un valor que no existe responde `400`.

Cambios de estado (`PATCH {"status": ...}` o `DELETE`, que equivale a `INACTIVO`):

- `INACTIVO` (baja): `400` si tiene recursos sin devolver (`ACTIVO` o `VENCIDO`).
- `SUSPENDIDO`: se permite aunque deba algo. Pierde el acceso y no puede pedir más; sus préstamos se siguen devolviendo en el mostrador.
- En ambos casos sus reservas sin recoger se cancelan y los ejemplares quedan libres.
- Nadie puede cambiar su propio rol ni su propio estado.

4. Catálogo de recursos

En la API, un **recurso** es una ficha ("Proyector Epson") y sus **ejemplares** son las unidades físicas, cada una con su código de inventario.

Listar recursos (cualquier usuario)

`GET /catalogo/recursos/`

Solo devuelve fichas `ACTIVA`, ordenadas por nombre (paginado: los datos van en `results`). Filtros opcionales, combinables:

- `?q=epson` — busca en nombre y descripción.
- `?categoria=<uuid>` — solo esa categoría (`400` si no es un UUID).
- `?solo_disponibles=true` — solo fichas con al menos una unidad libre.

```
{
  "count": 1, "next": null, "previous": null,
  "results": [
  {
    "id": "uuid",
    "nombre": "Laptop Dell XPS 15",
    "descripcion": "...",
    "categoria": "uuid o null",
    "categoria_nombre": "Equipos",
    "estado_ficha": "ACTIVA",
    "total_ejemplares": 3,
    "disponibles": 2,
    "disponibilidad": "DISPONIBLE"
  }
  ]
}
```

`total_ejemplares` no cuenta los ejemplares `DADO_DE_BAJA`. `disponibilidad` se calcula: `SIN_EJEMPLARES` si el total es 0, `DISPONIBLE` si hay al menos uno libre y `AGOTADO` si todos están prestados o en mantenimiento.

Ver un recurso (cualquier usuario)

`GET /catalogo/recursos/{id}/`

Mismos campos que en el listado. Una ficha dada de baja responde `404`.

Listar categorías (cualquier usuario)

`GET /catalogo/categorias/`

Para armar el filtro por categoría del catálogo.

Catálogo completo, con ejemplares (admin)

`GET /catalogo/admin/recursos/`

Incluye también las fichas `DADA_DE_BAJA` y un arreglo `ejemplares` por ficha. Acepta los mismos filtros que el listado público, más `?estado_ficha=ACTIVA` o `DADA_DE_BAJA`.

```
{
  "id": "uuid",
  "nombre": "Laptop Dell XPS 15",
  ...
  "ejemplares": [
    {"id": "uuid", "ficha_recurso": "uuid", "codigo_inventario": "LAP-001",
     "estado": "DISPONIBLE", "condicion": "BUENO", "observaciones": ""}
  ]
}
```

Crear una ficha (admin)

`POST /catalogo/admin/recursos/`

```
{"nombre": "Proyector Epson", "descripcion": "...", "categoria": "uuid"}
```

`categoria` es obligatoria. El nombre no se puede repetir dentro de la misma categoría (sin importar mayúsculas); si se repite responde `400`.

Ver, editar o eliminar una ficha (admin)

`GET, PATCH, DELETE /catalogo/admin/recursos/{id}/`

`PATCH` edita `nombre`, `descripcion` y `categoria`. `DELETE` borra la ficha y sus ejemplares **solo si ninguno se prestó nunca**; si hay historial responde `409` y hay que darla de baja.

Dar de baja o reactivar una ficha (admin)

`POST /catalogo/admin/recursos/{id}/baja/`

`POST /catalogo/admin/recursos/{id}/reactivar/`

Sin cuerpo. Responden la ficha actualizada, o `400` si ya estaba en ese estado. Una ficha de baja desaparece del catálogo público, no se puede reservar y no acepta ejemplares nuevos. Las reservas sin recoger de ese recurso se cancelan en el acto. Los préstamos ya entregados siguen y se devuelven normalmente.

Listar o registrar unidades físicas de una ficha (admin)

`GET, POST /catalogo/admin/recursos/{ficha_id}/ejemplares/`

```
{"codigo_inventario": "LAP-006", "condicion": "BUENO", "observaciones": "Nueva, en caja"}
```

- `codigo_inventario` se guarda sin espacios y en mayúsculas, y es único.
- `estado` es opcional: `DISPONIBLE` (por defecto) o `EN_MANTENIMIENTO`.
- `condicion` es opcional, `BUENO` por defecto.
- Responde `404` si la ficha no existe y `400` si está dada de baja.

Editar un ejemplar puntual (admin)

`GET, PATCH, DELETE /catalogo/admin/ejemplares/{id}/`

Reglas del `PATCH`:

- `estado` puede ser `DISPONIBLE`, `EN_MANTENIMIENTO` o `DADO_DE_BAJA` (perdido o roto).
- `PRESTADO` y `RESERVADO` los asigna solo el sistema: no se pueden poner a mano, y un ejemplar en esos estados no se cambia por `PATCH` (cambia al entregar, devolver o cancelar).
- Un ejemplar `DANADO` no puede quedar `DISPONIBLE`.
- No se puede cambiar de ficha.

`DELETE` solo funciona si el ejemplar nunca se prestó; si tiene historial responde `409` y hay que pasarlo a `DADO_DE_BAJA`.

5. Categorías (admin)

`GET, POST /catalogo/admin/categorias/`

`GET, PATCH, DELETE /catalogo/admin/categorias/{id}/`

```
{"nombre": "Equipos"}
```

`nombre` es único sin importar mayúsculas; repetirlo devuelve `400`. Eliminar una categoría que tiene fichas devuelve `409`.

6. Políticas de préstamo

`GET /prestamos/admin/politicas/`

`GET, PATCH /prestamos/admin/politicas/{id}/`

```
{"dias_maximo_prestamo": 10, "max_prestamos_simultaneos": 3}
```

- Hay una política por cada tipo de usuario (`TEACHER`, `STUDENT`, `ADMINISTRATIVE_STAFF`, `ADMINISTRATOR`) y vienen creadas de fábrica: no se crean ni se eliminan. `tipo_usuario` se ve pero no se cambia.
- `dias_maximo_prestamo` y `max_prestamos_simultaneos` deben ser al menos 1. `max_prestamos_simultaneos` es cuántos recursos puede tener afuera a la vez un usuario de ese tipo.
- Cambiar los días afecta a toda reserva **nueva** (y a las renovaciones); no recalcula las reservas ya existentes.
- Solo `PATCH`, no `PUT`.

7. Reservas

Ciclo de un préstamo

```
Desde la app: reservar -> RESERVADO --(admin: entregar/)--> ACTIVO
                            |
                            +--(usuario: cancelar/, o pasan 24 h)--> CANCELADO

En el mostrador (admin: POST /prestamos/admin/reservas/) -> ACTIVO de una vez

ACTIVO --(pasa la fecha limite)--> VENCIDO
ACTIVO o VENCIDO --(admin: devolver/)--> DEVUELTO
ACTIVO o VENCIDO --(admin: perdido/)---> PERDIDO
```

Reservar un recurso (usuario)

`POST /prestamos/reservar/`

```
{"ficha_recurso": "uuid"}
```

El backend busca un ejemplar `DISPONIBLE` de esa ficha (que debe estar `ACTIVA`) y lo aparta: el préstamo queda `RESERVADO` y el ejemplar `RESERVADO`. Todavía no tiene `fecha_limite`: los días empiezan a contar cuando lo recoge. `recoger_antes_de` indica hasta cuándo puede pasar por el mostrador (24 horas); después la reserva se cancela sola y el ejemplar se libera. Responde `201` con el préstamo completo (mismos campos que en *Ver mis préstamos*).

Una reserva sin recoger ya ocupa cupo: cuenta para `max_prestamos_simultaneos` y para la regla de un ejemplar por recurso.

Responde `400` con un mensaje de error si:

- no hay unidades libres o la ficha está dada de baja;
- el usuario tiene algún préstamo `VENCIDO` (primero debe devolverlo);
- ya tiene un ejemplar de ese mismo recurso sin devolver;
- llegó a su `max_prestamos_simultaneos`.

Ver mis préstamos (usuario)

`GET /prestamos/mis-reservas/`


```
{
  "count": 1, "next": null, "previous": null,
  "results": [
  {
    "id": "uuid",
    "usuario": 5,
    "usuario_email": "daniel@unsa.edu.pe",
    "tipo_usuario": "STUDENT",
    "ejemplar_fisico": "uuid",
    "codigo_inventario": "LAP-002",
    "ficha_recurso_nombre": "Laptop Dell XPS 15",
    "fecha_prestamo": "2026-10-06T09:30:00-05:00",
    "fecha_entrega": "2026-10-06T10:00:00-05:00",
    "fecha_limite": "2026-10-12",
    "fecha_devolucion": null,
```

```
    "estado": "ACTIVO",
    "recoger_antes_de": null,
    "con_retraso": false,
    "renovaciones": 0,
    "registrado_por_email": null
  }
  ]
}
```

- Ordenados del más reciente al más antiguo.
- `estado` se actualiza solo: un `ACTIVO` cuya `fecha_limite` ya pasó aparece como `VENCIDO`, y una reserva que pasó 24 horas sin recogerse aparece como `CANCELADO`.
- `fecha_prestamo`: cuándo se reservó (o se prestó en el mostrador). `fecha_entrega`: cuándo se lo llevó; `null` mientras está `RESERVADO`.
- `fecha_limite`: `null` mientras está `RESERVADO`.
- `recoger_antes_de`: solo en `RESERVADO`; si no, `null`.
- `con_retraso`: `true` si está vencido o si se devolvió después de la fecha límite. Sirve para marcar en rojo.
- `registrado_por_email`: el admin que lo entregó (al prestar en el mostrador o al marcar la entrega de una reserva); `null` mientras la reserva no se recoge.

Cancelar una reserva propia (usuario)

`POST /prestamos/mis-reservas/{id}/cancelar/`

Sin cuerpo. Para deshacer una reserva hecha por error: solo mientras está `RESERVADO`, es decir, **antes de recogerla**. Queda `CANCELADO` en el historial y el ejemplar vuelve a estar disponible. Si ya la recogió (`ACTIVO`, o un préstamo hecho en el mostrador) responde `400`: hay que devolverlo. El préstamo de otro usuario responde `404`.

Renovar un préstamo (admin)

`POST /prestamos/admin/reservas/{id}/renovar/`

Solo el admin: el usuario que necesita más días lo pide en el mostrador (no hay ruta para que lo haga desde la app). Sin cuerpo. La nueva `fecha_limite` es hoy más los días de la política, y `renovaciones` sube en 1. Responde `400` si:

- el préstamo no está `ACTIVO` (un `VENCIDO` no se renueva, se devuelve);
- ya se renovó una vez (máximo 1);
- el recurso fue dado de baja del catálogo;
- es muy pronto: la renovación no extendería el plazo (por ejemplo, el mismo día en que se pidió).

Listar todas las reservas (admin)

`GET /prestamos/admin/reservas/`

Filtros opcionales, combinables:

- `?estado=` `ACTIVO`, `VENCIDO`, `DEVUELTO`, `PERDIDO` o `CANCELADO`. Otro valor responde `400`.
- `?usuario=5` — historial de un usuario (por su `id`). Por ejemplo, `?usuario=5&estado=VENCIDO` muestra lo que ese usuario tiene atrasado.
- `?codigo_inventario=LAP-002` — por la etiqueta del equipo (se normaliza: `lap-002` también vale). Para devolver en el mostrador: `?codigo_inventario=LAP-002&estado=ACTIVO` trae el préstamo abierto de esa unidad (si está atrasado, con `estado=VENCIDO`). Sin estado, es el historial de esa unidad.
- `?ficha_recurso=<uuid>` — historial de un recurso con todas sus unidades (`400` si no es un UUID).

Registrar un préstamo en el mostrador (admin)

`POST /prestamos/admin/reservas/`

El admin presta a nombre de otro usuario. Se manda el recurso (el sistema elige la unidad) **o** el código de inventario de la unidad exacta que se está entregando, nunca los dos:

```
{"usuario": 5, "ficha_recurso": "uuid"}
{"usuario": 5, "codigo_inventario": "LAP-002"}
```

- `codigo_inventario` es el de la etiqueta del equipo: se puede escribir o leer con un lector de código de barras. Se normaliza igual que en el catálogo, así que `"lap-002 "` también vale.
- `usuario` es el `id`. Flujo sugerido: buscar a la persona con `GET /users/admin/usuarios/?q=<dni o cui>`, mostrar sus datos y lo que tiene pendiente (`GET /prestamos/admin/reservas/?usuario=<id>`) para que el admin confirme, y recién ahí hacer el `POST`.
- El préstamo nace `ACTIVO` (no pasa por `RESERVADO`): el usuario se lo lleva en ese momento. Por eso no se puede cancelar.
- Se aplican las mismas reglas que al reservar, y los días son los del tipo del usuario (no los del admin).
- También responde `400` si el usuario no está activo, si el código no existe o si ese ejemplar no está disponible (el mensaje indica su estado, por ejemplo *en mantenimiento*).

Ver una reserva puntual (admin)

`GET /prestamos/admin/reservas/{id}/`

Ya no acepta `DELETE` (responde `405`): el historial no se borra.

Entregar una reserva (admin)

`POST /prestamos/admin/reservas/{id}/entregar/`

Sin cuerpo. El usuario pasó por el mostrador a recoger lo que reservó desde la app. El préstamo pasa a `ACTIVO`, se guarda `fecha_entrega` y `fecha_limite` se calcula desde hoy con la política del usuario. El ejemplar pasa a `PRESTADO`. Desde aquí ya no se puede cancelar. Responde `400` si:

- no está `RESERVADO` (ya se entregó, se canceló o caducó por pasar las 24 horas);
- el usuario ya no está activo o tiene algún préstamo `VENCIDO`;
- el recurso se dio de baja del catálogo mientras estaba reservado.

Cancelar una reserva (admin)

`POST /prestamos/admin/reservas/{id}/cancelar/`

Sin cuerpo. Igual que la cancelación del usuario, pero sobre cualquier reserva (por ejemplo, si el alumno avisa que no va a ir). Solo en `RESERVADO`; si ya se entregó responde `400`: se registra la devolución.

Registrar la devolución (admin)

`POST /prestamos/admin/reservas/{id}/devolver/`

Cuerpo opcional, con la condición en la que vuelve el ejemplar:

```
{"condicion": "DANADO"}
```

Marca la reserva `DEVUELTO`, pone `fecha_devolucion` a hoy y libera el ejemplar: vuelve a `DISPONIBLE`, o pasa a `EN_MANTENIMIENTO` si vuelve `DANADO`. Un `VENCIDO` también se devuelve (queda con `con_retraso: true`). Una reserva ya cerrada (`DEVUELTO`, `PERDIDO` o `CANCELADO`) o todavía sin recoger (`RESERVADO`) responde `400`.

Marcar un recurso como perdido (admin)

`POST /prestamos/admin/reservas/{id}/perdido/`

Cuerpo opcional, con un detalle que se agrega a las observaciones del ejemplar:

```
{"observaciones": "Se lo robaron en el bus"}
```

Para cuando el recurso nunca vuelve. La reserva queda `PERDIDO` y el ejemplar pasa a `DADO_DE_BAJA` (deja de contar en el catálogo pero conserva su historial). No cuenta para el tope del usuario. Una reserva ya cerrada o todavía sin recoger (`RESERVADO`) responde `400`.

8. Estados posibles

Campo	Valores
`Prestamo.estado`	`RESERVADO`, `ACTIVO`, `VENCIDO` (pendientes); `DEVUELTO`, `PERDIDO`, `CANCELADO` (cerrados)
`EjemplarFisico.estado`	`DISPONIBLE`, `RESERVADO`, `PRESTADO`, `EN_MANTENIMIENTO`, `DADO_DE_BAJA`
`EjemplarFisico.condicion`	`BUENO`, `REGULAR`, `DANADO`
`FichaRecurso.estado_ficha`	`ACTIVA`, `DADA_DE_BAJA`

Campo	Valores
`disponibilidad` (calculado)	`DISPONIBLE`, `AGOTADO`, `SIN_EJEMPLARES`
`Usuario.tipo_usuario`	`TEACHER`, `STUDENT`, `ADMINISTRATIVE_STAFF`, `ADMINISTRATOR`